

Кросплатформне програмування

03

Базові типи даних і
основні оператори



Базові типи й оператори Java

Про що йтиметься:

- Базові типи даних
- Літерали і автоматичне приведення типів
- Арифметичні оператори
- Логічні оператори
- Оператори порівняння
- Побітові оператори
- Оператори присвоєння



Базові типи Java

Цілі числа

byte	8 біт
short	16 біт
int	32 біт
long	64 біт

Дійсні числа

float	32 біт
double	64 біт

Символи

char	16 біт
-------------	---------------

Логічний тип

boolean	значення true або false
----------------	---------------------------------------

Літерали

125	int
125L	long
0b101	2-ковий
012	8-ковий
0x12	16-ковий
2.0	double
2.0F	float

Використовуємо:
int double char

Змінні й автоматичне приведення типів

Оголошення змінних:

- Вказується ідентифікатор типу й ім'я змінної
- Оголошення у будь-якому місці, але до першого використання
- Область доступності визначається блоком, в якому оголошена змінна (блок виділяється дужками { та })

Автоматичне приведення типів:

- Тип **char** переводиться в **int**
- Типи **byte** і **short** перетворюються в **int**
- Якщо є **long** - все перетворюється в **long**
- Якщо є **float** - все перетворюється в **float**
- Якщо є **double** - все перетворюється в **double**



- Типи мають бути сумісними
- Цільовий тип "більше" вихідного
- Автоматичне приведення об'єктів до String

Змінні: приклади

ПОМИЛКА!!!

```
byte a=50;  
a=a+1;
```



Літерал **1** має тип **int**

Правильно!!!

```
byte a=50;  
a=(byte) (a+1);
```



Явне приведення типу

ПОМИЛКА!!!

```
byte a=5,b=1,c;  
c=a+b;
```



Сума **byte**-змінних
розраховується як **int**-значення

Правильно!!!

```
byte a=5,b=1,c;  
c=(byte) (a+b);
```



Явне приведення
типу

Оператори в Java

Основні групи операторів:

- Арифметичні оператори
- Логічні оператори
- Оператори порівняння
- Побітові оператори



А також:

- Оператор присвоєння (повертає значення)
- Скорочені форми операцій присвоєння
- Тернарний оператор



Арифметичні оператори

Оператор	Дія
+	додавання
-	віднімання
*	множення
/	ділення
%	залишок від ділення
++	інкремент
--	декремент

Для цілочислових операндів виконується цілочислове ділення: результатом виразу $8/5$ є 1. Для звичайного ділення використовують команду `(double) 8/5` (результат дорівнює 1.6)

Під час виконання команди виду `a++` (або `++a`) змінна `a` збільшує значення на 1. Під час виконання команди виду `a--` (або `--a`) змінна `a` зменшує значення на 1

```
int a=10,b;  
b=a++;  
// a=11, b=10
```

```
int a=10,b;  
b=++a;  
// a=11, b=11
```



Логічні оператори

Оператор	Дія
&	логічне і
&&	логічне і (скорочена форма)
	логічне або
	логічне або (скорочена форма)
^	виняткове або
!	логічне заперечення

Логічне і: команда виду $A \& B$

A \ B	true	false
true	true	false
false	false	false

Логічне або: команда виду $A | B$

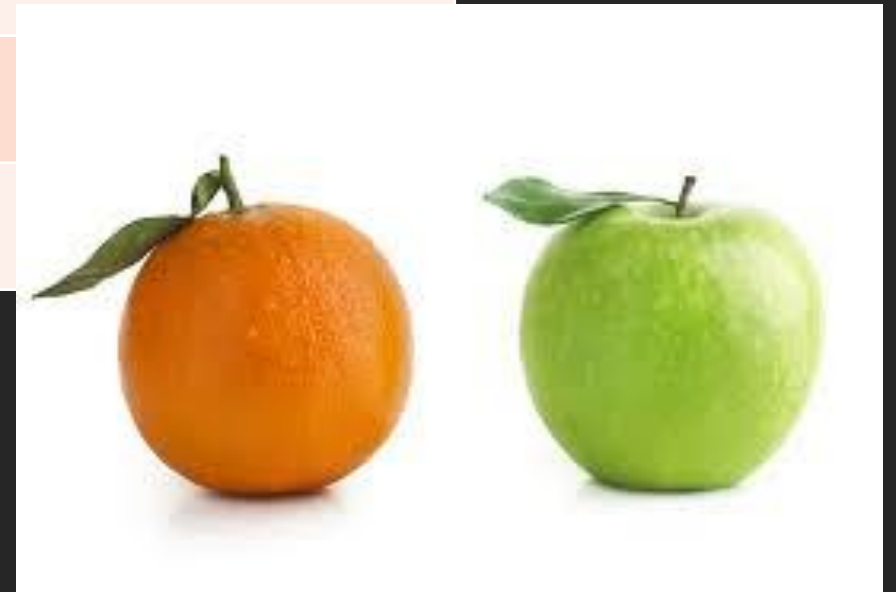
A \ B	true	false
true	true	true
false	true	false

Виняткове або: команда $A \wedge B$

A \ B	true	false
true	false	true
false	true	false

Оператори порівняння

Оператор	Дія
<	менше
<=	менше або дорівнює
>	більше
>=	більше або дорівнює
==	дорівнює
!=	не дорівнює





Системи числення

- Кількість використовуваних цифр визначає систему числення

- Якщо використовується h цифр, то в числі $\overline{b_m b_{m-1} \dots b_2 b_1 b_0}$

параметри $b_k = 0, 1, 2, \dots, h - 1$

- В десятковій системі

$$\overline{b_m b_{m-1} \dots b_2 b_1 b_0} = b_0 \cdot h^0 + b_1 \cdot h^1 + b_2 \cdot h^2 + \dots + b_m \cdot h^m$$

- Для двійкової системи $h = 2$

$$\overline{b_m b_{m-1} \dots b_2 b_1 b_0} = b_0 \cdot 2^0 + b_1 \cdot 2^1 + b_2 \cdot 2^2 + \dots + b_m \cdot 2^m$$

параметри $b_k = 0, 1$

- Двійковий код 1101 відповідає числу

$$1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 13$$

- Для вісімкової системи $h = 8$

$$\overline{b_m b_{m-1} \dots b_2 b_1 b_0} = b_0 \cdot 8^0 + b_1 \cdot 8^1 + b_2 \cdot 8^2 + \dots + b_m \cdot 8^m$$

параметри $b_k = 0, 1, 2, \dots, 7$

- Вісімковий код 123 відповідає числу

$$1 \cdot 8^2 + 2 \cdot 8^1 + 3 \cdot 8^0 = 64 + 16 + 3 = 83$$



Шістнадцяткова система

- В шістнадцятковій системі ($h = 16$) крім десяти цифр від 0 до 9 ще використовуються букви від A (позначає число 10) до F (позначає число 15)

- Переведення у десяткову систему виконується відповідно до співвідношення

$$\overline{b_m b_{m-1} \dots b_2 b_1 b_0} = b_0 \cdot 16^0 + b_1 \cdot 16^1 + b_2 \cdot 16^2 + \dots + b_m \cdot 16^m$$

параметри $b_k = 0, 1, 2, \dots, 15$

- Наприклад, шістнадцятковий код $A1B$ відповідає числу

$$10 \cdot 16^2 + 1 \cdot 16^1 + 11 \cdot 16^0 = 2560 + 16 + 11 = 2587$$

(в шістнадцятковому представленні символ A відповідає числу 10, а символ B відповідає числу 11)



Двійкове кодування

- В двійковому представленні число записується як послідовність нулів та одиниць.

- Використовується n бітів (значення 0 або 1).

- Бінарний код числа $\overline{a_{n-1}a_{n-2} \dots a_2a_1a_0}$.

- В десятковій системі

$$a_0 \cdot 2^0 + a_1 \cdot 2^1 + a_2 \cdot 2^2 + \dots + a_{n-2} \cdot 2^{n-2} + a_{n-1} \cdot 2^{n-1}$$

- Операції з числами у двійковій системі:

$$0 + 0 = 0$$

$$1 + 0 = 1$$

$$0 + 1 = 1$$

$$1 + 1 = 10$$



Від'ємні числа

- Маємо деяке додатне число x
- Необхідно згенерувати код для числа $-x$
- За визначенням $-x + x = 0$
- До числа x застосовуємо побітову інверсію: одиниці замінюємо на нулі, а нулі міняємо на одиниці. Результат позначимо як $\sim x$
- Результатом виразу $\sim x + x$ є код $\underbrace{111 \dots 11}_n$ з n одиниць
- Прибавимо число 1 (розраховуємо суму $\sim x + x + 1$): отримаємо код $1 \underbrace{000 \dots 00}_n$, в якому n нулів і у старшому біті 1
- Комп'ютер запам'ятовує n бітів і старший одиничний біт втрачається
- Отримуємо код з n нулів: має місце співвідношення $\sim x + x + 1 = 0$
- Вираз $\sim x + 1$ дає код від'ємного числа $-x$

Від'ємні числа

- В двійковому представленні числа старший біт називається знаковим і визначає знак числа: у додатних чисел старший біт нульовий, а у від'ємних чисел старший біт одиничний.
- Щоб за кодом додатнього числа отримати код протилежного йому від'ємного числа, необхідно інвертувати код додатного числа і додати до результату одиницю.
- Щоб за кодом від'ємного числа визначити це число в десятковій системі числення, необхідно виконати інверсію колу від'ємного числа, прибавити до коду одиницю, перевести отриманий код у десяткове значення (як це робиться для додатніх чисел) і "дописати" перед числом знак "мінус".



Побітові оператори

Оператор	Дія
&	побітове і
	побітове або
^	побітове виняткове або
~	побітова інверсія
<<	зсув вліво
>>	зсув вправо
>>>	зсув вправо із заповненням старшого біту нулями

Number 1	1	0	1	0	1
Number 2	1	1	1	0	0

AND	1	0	1	0	0
OR	1	1	1	0	1
XOR	0	1	0	0	1

Побітові оператори

Побітове і: $a \& b$

$a \backslash b$	1	0
1	1	0
0	0	0

Побітове або: $a | b$

$a \backslash b$	1	0
1	1	1
0	1	0

Виняткове або: $a \wedge b$

$a \backslash b$	1	0
1	0	1
0	1	0

Інверсія: $\sim a$

a	0	1
$\sim a$	1	0

Побітові оператори

Зсув вліво <<

`A << n`



Побітове представлення змінної **A** зсувається вліво на **n** позицій. Старші біти втрачаються, молодші заповнюються нулями

Зсув вправо >>

`A >> n`



Побітове представлення змінної **A** зсувається вправо на **n** позицій. Молодші біти втрачаються, старші заповнюються значенням знакового біту

Зсув вправо >>>

`A >>> n`



Побітове представлення змінної **A** зсувається вправо на **n** позицій. Молодші біти втрачаються, старші заповнюються нулями

Оператори

- Оператор присвоєння повертає значення

```
int x,y,z;  
x=y=z=100;
```



Всі три змінні отримують значення 100

- Команда виду $A = A \odot B$ може бути записана як $A \odot = B$ (тут $\odot$ - бінарний арифметичний або побітовий оператор)

Наприклад

$A = A + B$



$A += B$

$A = A * B$



$A *= B$

$A = A ^ B$



$A ^= B$

Тернарний оператор

- Тернарний оператор:
умова?значення_1:значення_2

```
num = (x < 0) ? 0 : 1;
```



Змінна **num** отримує значення **0** якщо **x** менше **0**, та **1** в протилежному випадку

Тернарний оператор повертає результат!



Пріоритет операторів

1. () [] .

8. &

2. ++ -- ~ !

9. ^

3. * / %

10. |

4. + -

11. &&

5. >> << >>>

12. ||

6. > >= <= <

13. ?:

7. == !=

14. = op=





Приклад: парність числа

```
import static javax.swing.JOptionPane.*;
class Demo{
    public static void main(String[] args){
        int number,remainder;
        number=Integer.parseInt(
            showInputDialog("Уведіть ціле число:")
        );
        // Залишок від ділення на 2:
        remainder=number%2;
        String txt="Ви ввели ";
        // Використаний тернарний оператор:
        txt+=(remainder==0?"парне":"непарне")+" число!";
        showMessageDialog(null,txt);
    }
}
```

Приклад: парність числа

Input

Уведіть ціле число:

124

OK Cancel



Message

Ви ввели парне число!

OK

Input

Уведіть ціле число:

123

OK Cancel



Message

Ви ввели непарне число!

OK



Приклад: кількість сотень

```
import static javax.swing.JOptionPane.*;
class HundredsDemo{
    public static void main(String[] args){
        int number,hundreds;
        number=Integer.parseInt(
            showInputDialog("Уведіть ціле число:")
        );
        // Кількість сотен у числі (для цілочислових
        // операндів ділення виконується націло):
        hundreds=( (number/10)/10)%10;
        String txt="В цьому числі "+hundreds+" сотень!";
        showMessageDialog(null,txt);
    }
}
```

Приклад: кількість сотень

Input

Уведіть ціле число:

1234567

OK Cancel



Message

В цьому числі 5 сотень!

OK

Input

Уведіть ціле число:

12

OK Cancel



Message

В цьому числі 0 сотень!

OK

Дякую за увагу!

Запитання?